

Compressing Large Language Models

U Kang
Head Professor, IPAI
Professor, Dept. of CSE
Seoul National University

Outline

- ➡ ☐ LLM Compression
 - ☐ PTQ & QAT
 - ☐ Q-Lora & QA-Lora

Language Models

- Language models show astonishing improvements in natural language processing
 - Code generation
 - Text summarization
 - Chat
 - Translation
 - Content generation



Hi, could you write a code for a 2-layered CNN for CIFAR-100 using PyTorch?



Certainly! Here's a PyTorch code for defining a 2-layered Convolutional Neural Network (CNN) for the CIFAR-100 dataset:

```
python Copy code

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

# Define the 2-layer CNN model
class TwoLayerCNN(nn.Module):
    def __init__(self):
        super(TwoLayerCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, padding=1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.fc = nn.Linear(128 * 8 * 8, 100) # 100 classes in CIFAR-100

    def forward(self, x):
        x = self.pool1(self.relu1(self.conv1(x)))
        x = self.pool2(self.relu2(self.conv2(x)))
        x = x.view(-1, 128 * 8 * 8)
        x = self.fc(x)
        return x
```

Language Models

- However, language models are too large and generate severe side effects
 - Examples of language models with their sizes
 - OPT-175B, LLaMA-65B, and LLaMA2-70B
 - Side effects
 - Huge storage (memory, disk) requirement
 - Computationally expensive
 - Uses lots of energy
 - Hard to deploy models on small devices (e.g., smart phones)
- We need to compress language models to benefit language models without side effects

Language Model Compression

- Goal: compress a language model to be fast, memory-efficient, and energy-efficient while maintaining the accuracy
- Practical constraints
 - The task is to compress an accurate *pretrained* model which is trained on an extensive corpus
 - It is important to minimize the cost of compression algorithms considering the massive scale of language models

Compression Methods

- Pruning
- Distillation
- Quantization

Pruning

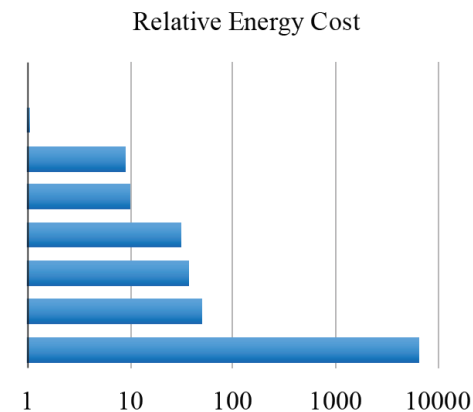
■ Standard deep neural network has...

- ❑ Significant memory usage
- ❑ Heavy power consumption

Input: Deep neural network to train

Output: Compressed deep neural network which preserves accuracy

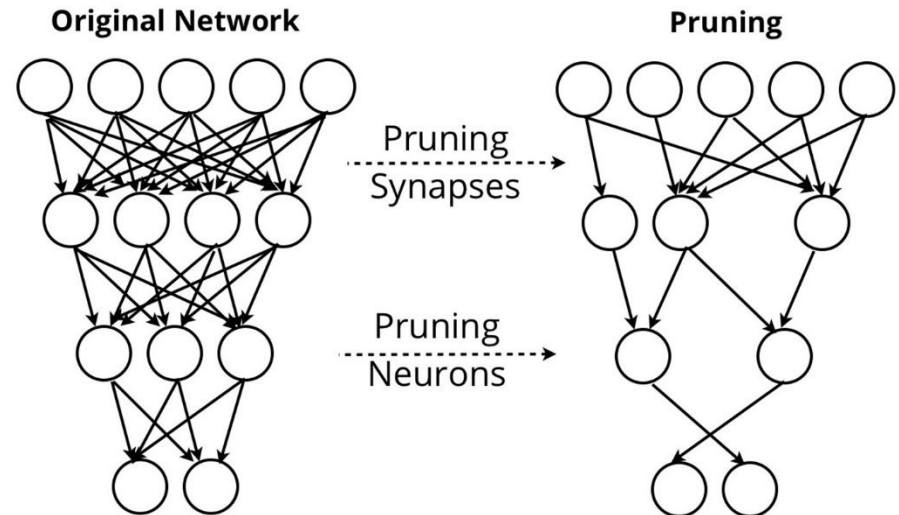
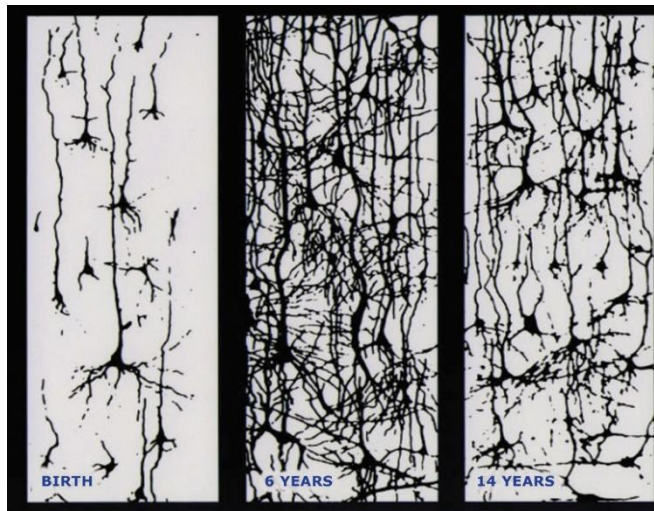
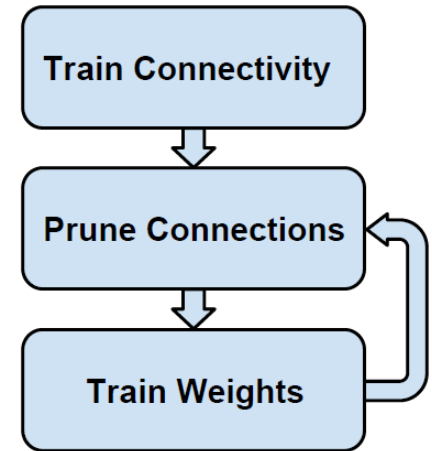
Operation	Energy [pJ]	Relative Cost
32 bit int ADD	0.1	1
32 bit float ADD	0.9	9
32 bit Register File	1	10
32 bit int MULT	3.1	31
32 bit float MULT	3.7	37
32 bit SRAM Cache	5	50
32 bit DRAM Memory	640	6400



Pruning

■ Pruning Weights

- Motivated by how real brain learns
- **Remove** weights which $|weight| < threshold$
- **Retrain** after pruning weights
- Learn effective connections by iterative pruning



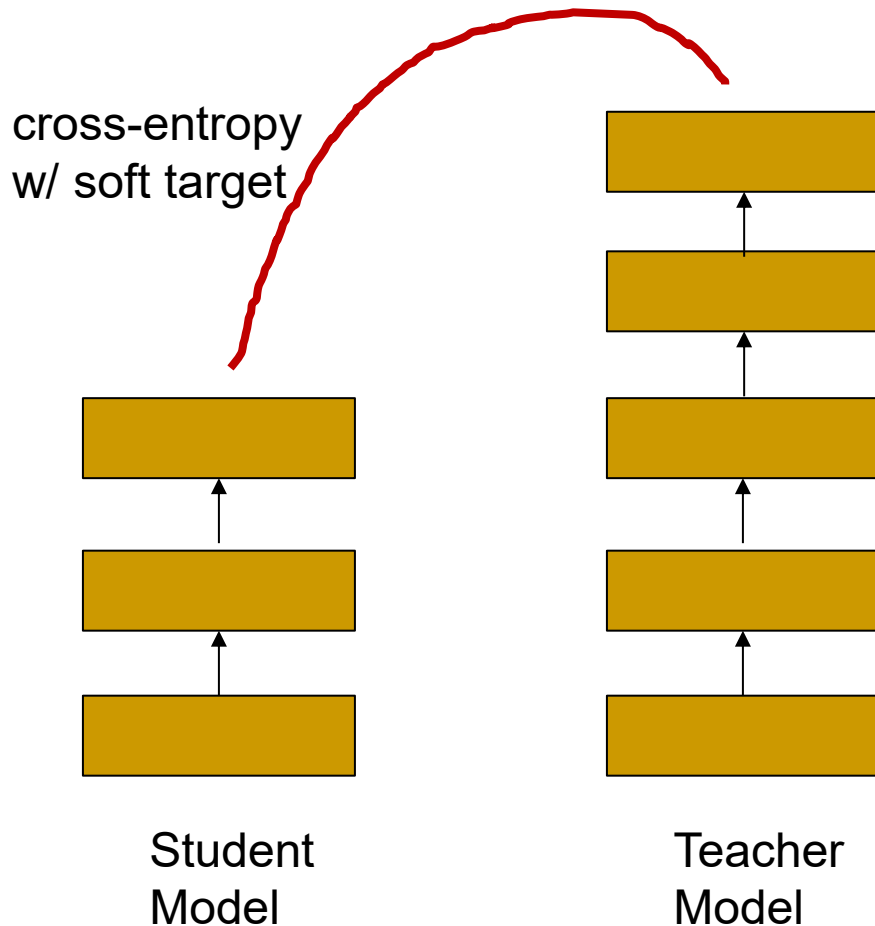
Pruning

- Compressing CNN by pruning
 - Experimented with LeNet (MNIST), AlexNet, VGGNet (ImageNet)
 - About 10x compared to original network

Network	Top-1 Error	Top-5 Error	Parameters	Compression Rate
LeNet-300-100 Ref	1.64%	-	267K	
LeNet-300-100 Pruned	1.59%	-	22K	12×
LeNet-5 Ref	0.80%	-	431K	
LeNet-5 Pruned	0.77%	-	36K	12×
AlexNet Ref	42.78%	19.73%	61M	
AlexNet Pruned	42.77%	19.67%	6.7M	9×
VGG-16 Ref	31.50%	11.32%	138M	
VGG-16 Pruned	31.34%	10.88%	10.3M	13×

Knowledge Distillation

■ Based on teacher-student model



Making soft target:

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Large T makes softer distribution

The small model is trained with the weighted average of

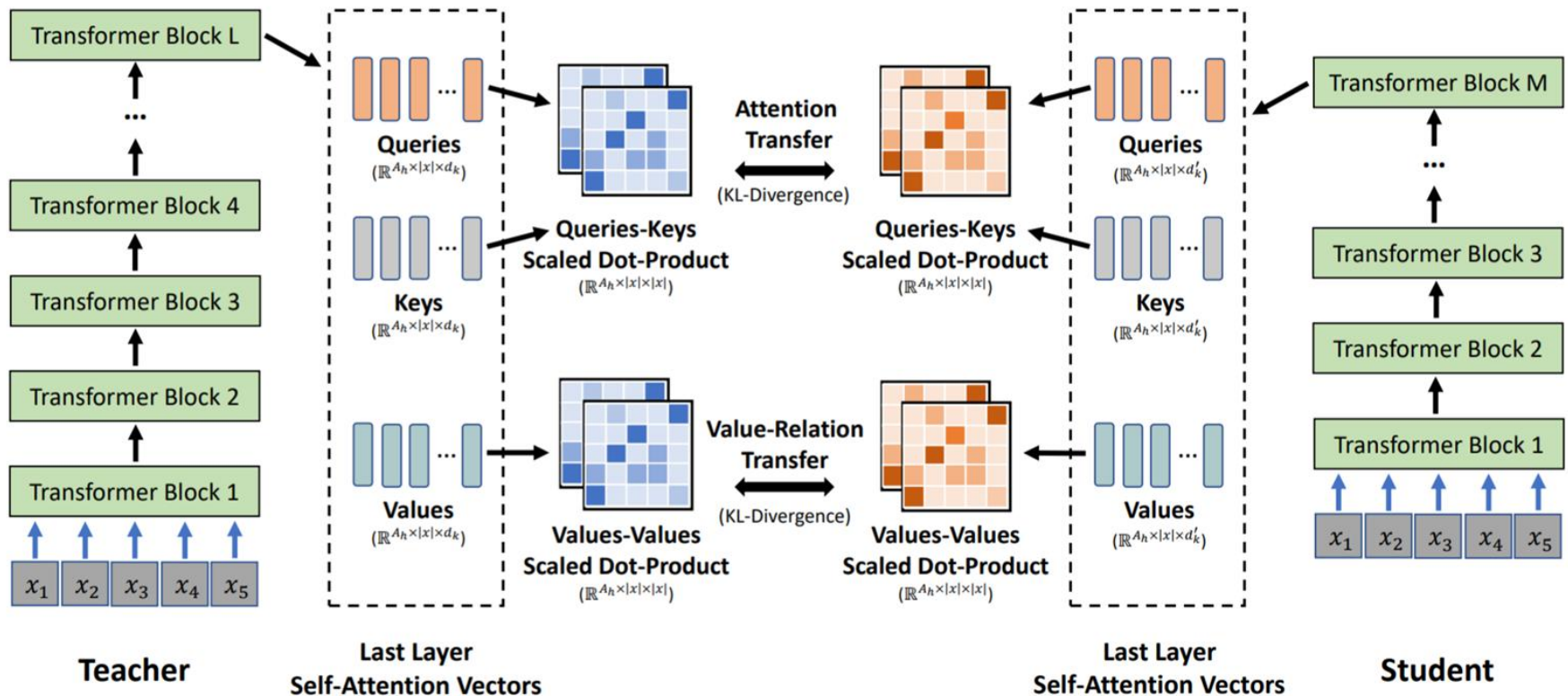
- cross-entropy w/ teacher's soft target
- cross-entropy w/ correct labels

Knowledge Distillation

- Given a BERT model, compress it to a lightweight one while maintaining its accuracy
- Teacher = Original BERT Model
- Student = Compressed BERT Model

Knowledge Distillation

■ Deep Self-Attention Distillation



[Wang et al., MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers, NeurIPS 2020]

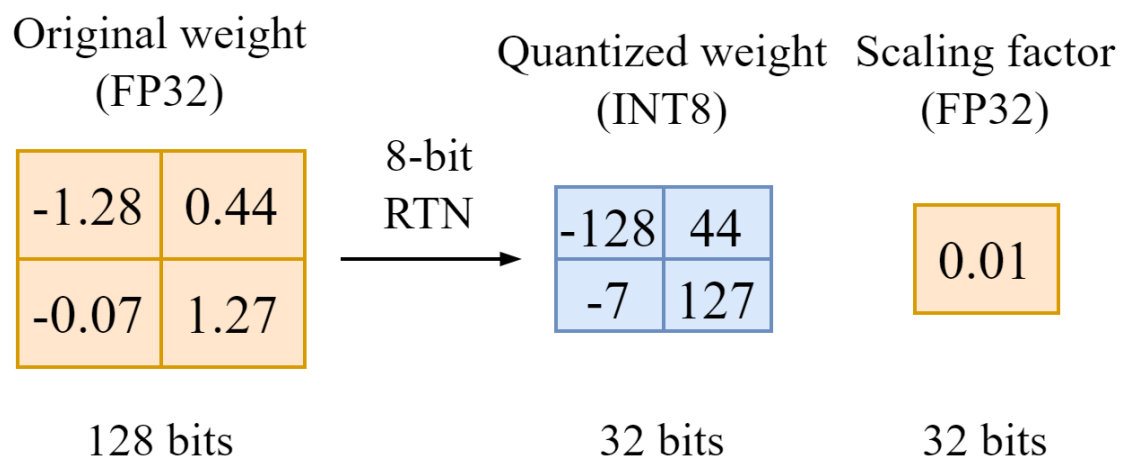
Knowledge Distillation

- Results on SQuAD 2.0 and GLUE dev sets
 - Comparison to the state-of-the-art distillation methods
 - MiniLM gives the best performance

Model	#Param	SQuAD2	MNLI-m	SST-2	QNLI	CoLA	RTE	MRPC	QQP	Average
BERT _{BASE} [12] (teacher)	109M	76.8	84.5	93.2	91.7	58.9	68.6	87.3	91.3	81.5
BERT _{SMALL} [41]	66M	73.2	81.8	91.2	89.8	53.5	67.9	84.9	90.6	79.1
Truncated BERT _{BASE} [12]	66M	69.9	81.2	90.8	87.9	41.4	65.5	82.7	90.4	76.2
DistilBERT [35]	66M	70.7	82.2	91.3	89.2	51.3	59.9	87.5	88.5	77.6
TinyBERT [20]	66M	73.1	83.5	91.6	90.5	42.8	72.2	88.4	90.6	79.1
MiniLM	66M	76.4	84.0	92.0	91.0	49.2	71.5	88.4	91.0	80.4

Uniform Quantization

- Uniform quantization maps real values to their corresponding b -bit integer values
 - b is the bit-width of the quantized model; generally $b \leq 8$
 - Round-to-nearest (RTN) is the basic approach of uniform quantization



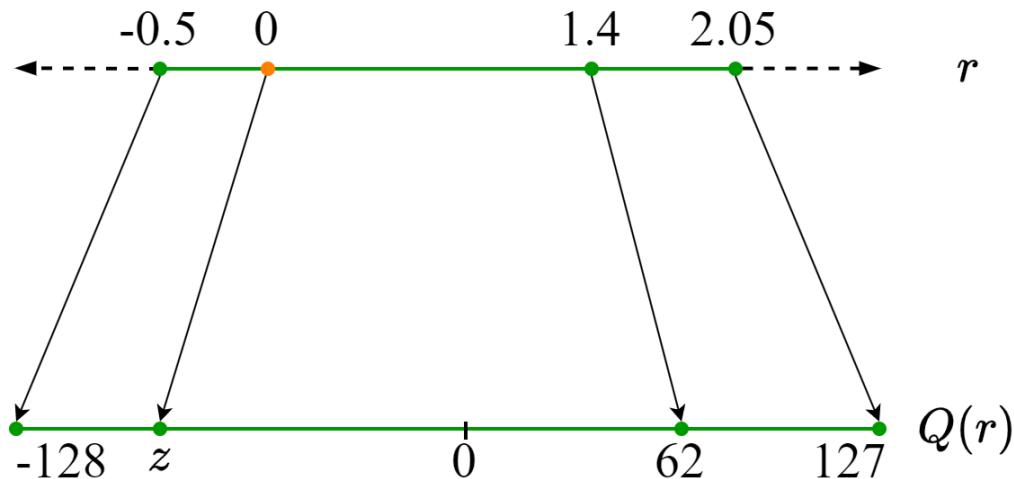
Round-to-nearest (RTN)

■ Quantization function of RTN

$$Q(r) = \left\lfloor \frac{r}{\Delta} \right\rfloor + z$$

where $\Delta = \frac{\max r - \min r}{2^b - 1}$ and $z = \left\lfloor -\frac{2^{b-1} \cdot \max r + (2^{b-1} - 1) \cdot \min r}{\max r - \min r} \right\rfloor$

- Δ is a scaling factor
- z is an integer value corresponding to zero input (i.e., $Q(0) = z$)



In this case,

$$\Delta = \frac{2.05 + 0.5}{2^8 - 1} = \frac{2.55}{255} = 0.01$$

$$z = \left\lfloor -\frac{128 \cdot 2.05 - 127 \cdot 0.5}{2.05 + 0.5} \right\rfloor = -78$$

When $r = 1.4$,

$$Q(r) = \left\lfloor \frac{1.4}{0.01} \right\rfloor - 78 = 62$$

Quantization Targets

- When we quantize a given neural network model, we can quantize:
 - (Weight-only quantization) only weights of the given model
 - (Activation quantization) both weights and activations of the given model
- Both methodologies reduce the bit-width to store the model
- Activation quantization has faster inference speed than weight-only quantization since activation quantization does not require ***dequantization***

Dequantization

- Dequantization recovers real values from the quantized values
- A recovered real value $\tilde{r}$ is calculated as follows:

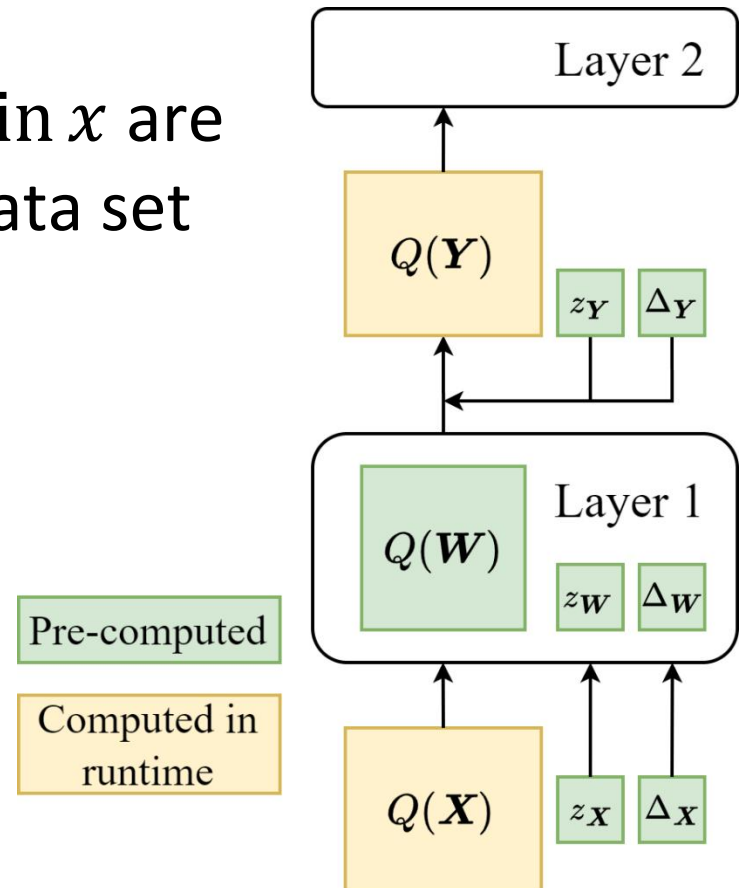
$$\tilde{r} = \Delta \cdot (Q(r) - z)$$

- Recap: $Q(r) = \left\lfloor \frac{r}{\Delta} \right\rfloor + z$

- In the case of weight-only quantization, quantized weights should be dequantized before multiplied with full-precision activations

Activation Quantization

- Quantize both weights and activations of a model
- Since the activations differ for each input data, $\max x$ and $\min x$ are pre-calculated using sample data set (i.e., calibration)



Outline

☒ LLM Compression

➔ ☐ **PTQ & QAT**

➔ **PTQ Methods**

QAT Methods

☐ Q-Lora & QA-Lora

PTQ vs QAT

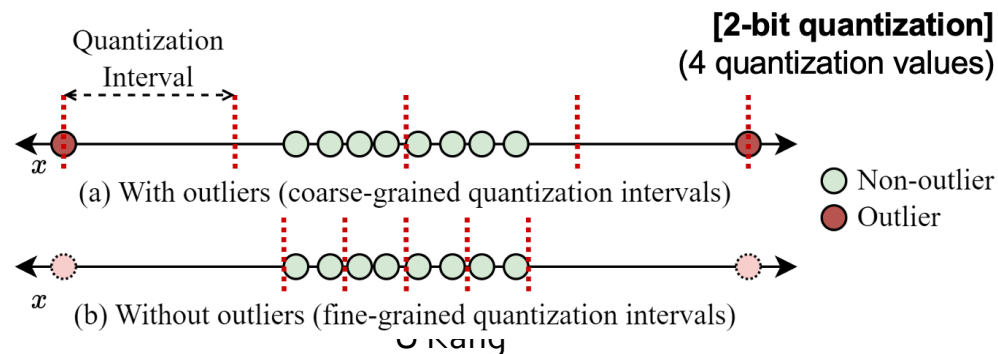
- Algorithms that entail end-to-end model retraining are extremely expensive for quantizing LLMs
- Post-training quantization (PTQ): does not require model retraining
 - Less accurate, but efficient
- Quantization-aware training (QAT): requires model retraining
 - More accurate, but inefficient

SmoothQuant

- **Title:** Accurate and Efficient Post-Training Quantization for Large Language Models [ICML' 23]
- Activations are harder to quantize than weights since they contain outliers that induce the quantization error
- SmoothQuant moves the magnitude of outlier channels in activation to the corresponding weights to reduce the difficulty of quantizing activation

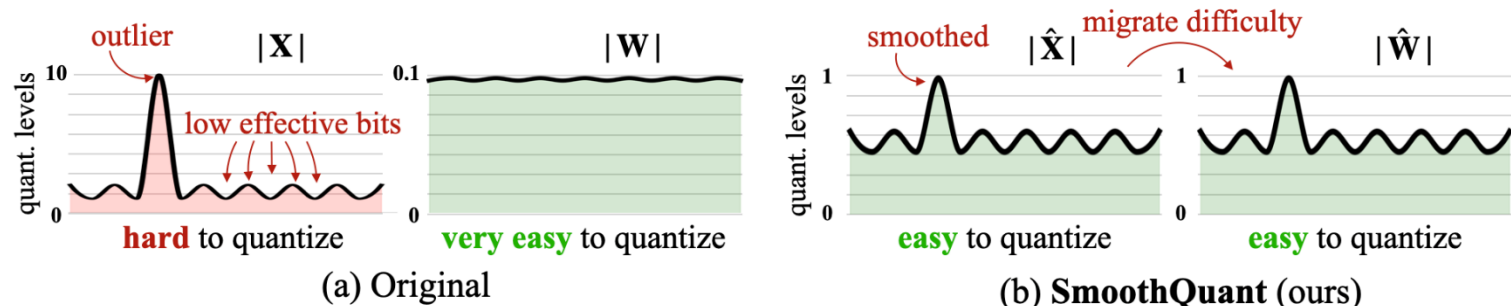
Challenge

- *How can we mitigate the difficulty of quantizing activations that contain outliers?*
- Activations in LLMs contain large outliers in specific channels
- Outliers significantly degrade the performance of the quantized models by widening the quantization intervals
 - This makes quantizing activations much harder than quantizing weights



Main Idea

- *Smooth the activation outliers by migrating the quantization difficulty from activations to weights*
 - Scale down outlier channels in activations and scale up corresponding weights to mitigate outliers
- $WX \approx Q(W \text{diag}(\mathbf{s}))Q(\text{diag}(\mathbf{s})^{-1}X)$, where W is weight, X is activation, and $\mathbf{s}$ is migration parameter
 - $s_j = \max(|X_j|)^\alpha / \max(|W_j|)^{1-\alpha}$, where α is a balancing hyperparameter to evenly distribute the difficulties



QuaRot

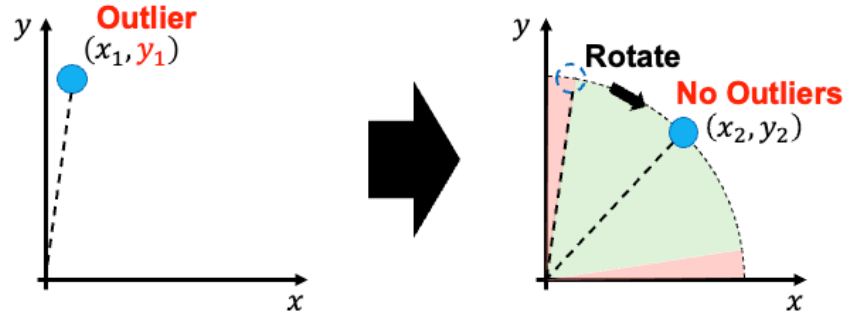
- **Title:** Outlier-Free 4-Bit Inference in Rotated LLMs [NeurIPS '24]
- Rotating a vector in random direction distributes its magnitude in all dimensions and removes an outlier
- QuaRot quickly rotates each activation using randomized Hadamard matrix and restores it after matrix multiplication to mitigate quantization error induced by outliers

Challenge

- *How can we remove the outliers of activations during quantization?*
- Activations contain outliers which increase the quantization error
- Removing outliers without altering the result of matrix multiplication would improve the quantization performance

Main Idea

- *Rotating an activation before quantization to remove the outlier*
- Rotating a vector in the random direction probabilistically removes outliers



- QuaRot utilizes randomized Hadamard matrix which enables the fast Hadamard transform for a quick random rotation
 - $WX \approx Q(WH^{-1})Q(HX)$, where H is a randomized Hadamard matrix

Outline

☒ LLM Compression

➔ ☐ **PTQ & QAT**

PTQ Methods

➔ **QAT Methods**

☐ Q-Lora & QA-Lora

QAT

- **Title:** EfficientQAT: Efficient Quantization-Aware Training for Large Language Models [ACL '25]
 - Current SOTA in QAT of LLMs
 - Better performance compared to PTQ with affordable amount of computational overhead (even under one A100 GPU constraint)
 - Weight-only quantization
 - **Challenge:** Efficient QAT (1. fast training, 2. accurate training)

Method	One A100-80GB?	GPU hours (h)
LLM-QAT	✗	900
QuiP#	✗	300
AQLM	✓	336
BitDistiller	✗	64
PB-LLM	✗	90
DB-LLM	✗	82
EfficientQAT	✓	41

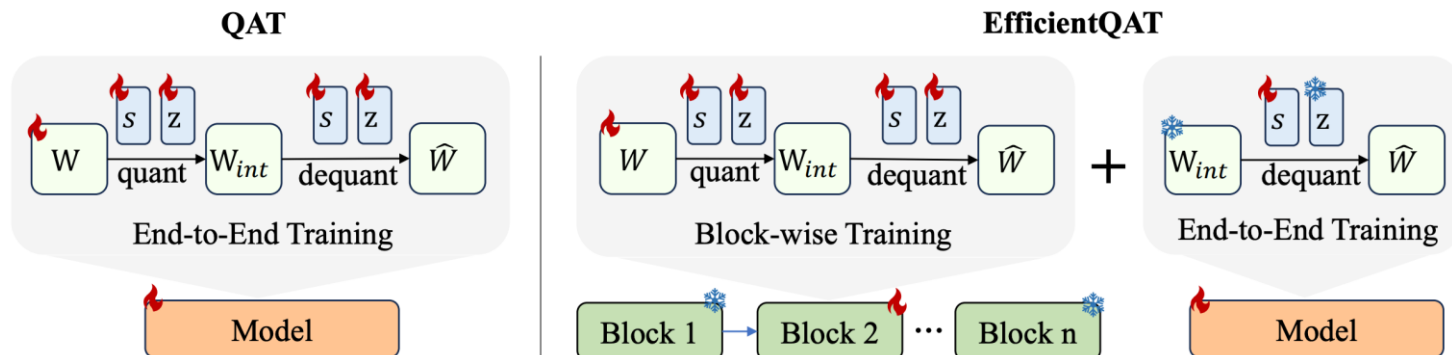
Llama-2	Block-AP		E2E-QP		Total Time
	Time	Memory	Time	Memory (4-/3-/2-bits)	
7B	3.3h	8.5GB	~1.5h	7.0/6.4/5.6GB	4.8h
13B	5.6h	10.3GB	~2.9h	11.7/10.6/9.1GB	8.5h
70B	26.6h	29.9GB	~14.3h	48.4/42.0/34.2GB	40.9h

QAT

■ EfficientQAT [ACL '25]

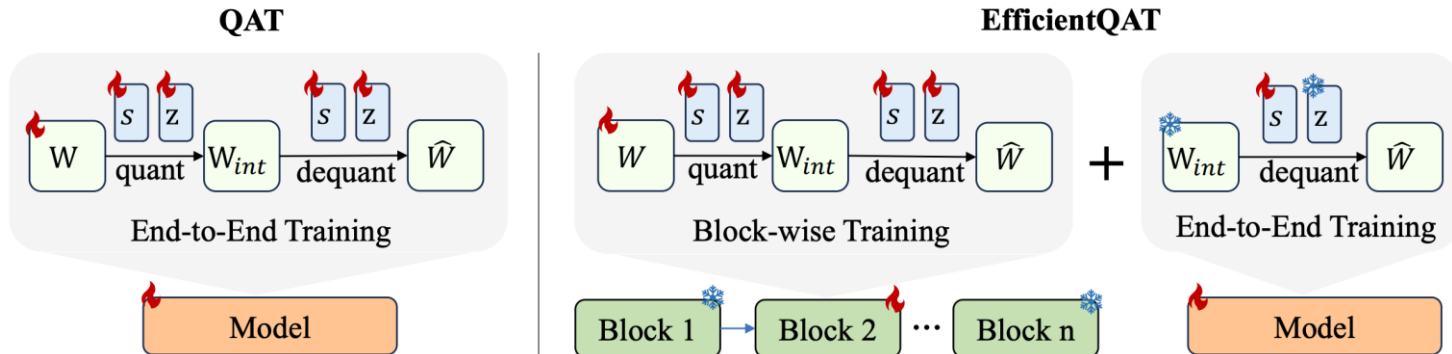
- **How?** Two-stage approach
- 0. RTN (Round-To-Nearest)
- 1. Block-AP: Sequential block-wise training of all parameters (s, z, W)
- 2. E2E-QP: End-to-end training of quantization parameters (s)
- Existing QAT (E2E-AP): End-to-end training of all parameters

** scaling factor s , zero-point z , and weight matrix W



QAT

- EfficientQAT [ACL '25]
 - **Why does it work?** Block-AP provides both efficient and effective initialization for the following end-to-end training (E2E-QP)
 - ➔ Total of 41 hours to quantize a ~70B model with QAT



Outline

☒ LLM Compression

☒ PTQ & QAT

 ☐ Q-Lora & QA-Lora

PEFT

- Existing high-cost compression algorithms for small language models show superior performance through repeated fine-tuning
 - Fine-tuning updates all parameters of the LLMs using a *task-specific* objective function and data
- However, LLMs are too large to fine-tune the entire model
 - GPT-3 175B requires 1.2TB VRAM to update the entire model
 - An 80GB A100 GPU costs more than 15,000\$
- How can we effectively fine-tune massive-scale LLMs?

PEFT

- PEFT (**P**arameter **E**fficient **F**ine-**T**uning) is an efficient algorithm to fine-tune LLMs
 - When fine-tuning a pre-trained LLM using PEFT, only a small number of (additional) model parameters are modified.
 - PEFT allows previous accurate high-cost algorithms to be applied for LLMs

LoRA

- LoRA (**Low-Rank Adaptation**) leverages low-rank approximation to reduce the cost of fine-tuning
- Overview of LoRA
 - (1) Parameter-efficient fine-tuning
 - LoRA divides the fine-tuned parameters into pre-trained parameters and update target parameters
 - (2) Low-rank approximation
 - LoRA applies low-rank approximation to update target parameters

LoRA

- LoRA reformulates the fine-tuning process

- Standard full fine-tuning

$$\mathcal{L}(X, y; \Theta_0) \rightarrow \mathcal{L}(X, y; \Theta)$$

- Fine-tuning in a parameter-efficient approach

$$\mathcal{L}(X, y; \Theta_0) \rightarrow \mathcal{L}(X, y; \Theta_0 + \Delta\Theta)$$

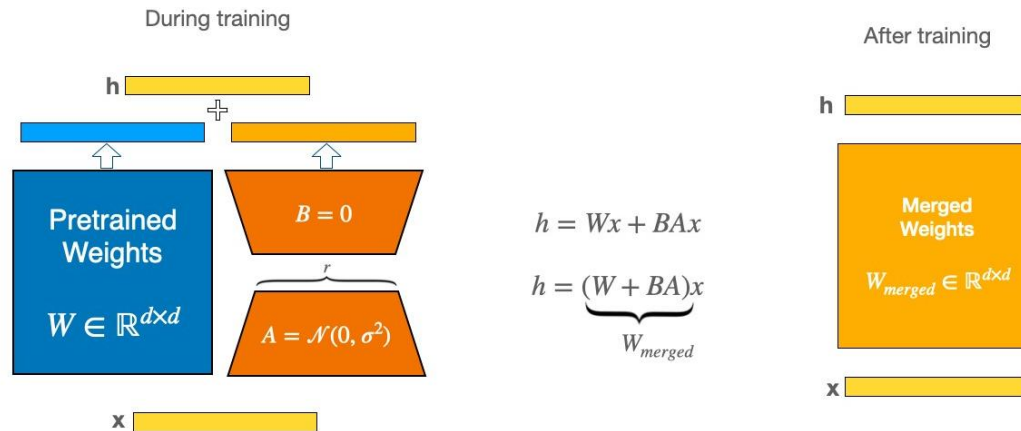
- $\mathcal{L}$ is the task-specific loss function
 - X and y represent the data and its corresponding label, respectively
 - Θ refers to the fine-tuned parameters
 - Θ_0 and $\Delta\Theta$ are the parameters from the pre-trained model and the update target parameters, respectively

LoRA

- LoRA applies low-rank approximation to the update target weight $\Delta W \in \mathbb{R}^{d \times d}$, not to the pretrained weight $W \in \mathbb{R}^{d \times d}$

$$h = Wx + \Delta Wx = Wx + BAx$$

- $x \in \mathbb{R}^d$ is an input and $h \in \mathbb{R}^d$ is an output
- $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ are low-rank matrices to approximate ΔW
 - r is a rank of low-rank approximation



- LoRA adapts only the MHA sublayer $W \in \{W_q, W_k, W_v, W_o\}$

LoRA

- The most significant benefit of LoRA comes from the reduction in memory footprint
 - On GPT-3 175B, LoRA reduces the VRAM consumption during training from 1.2TB to 350GB
 - We do not need to store the optimizer states for the frozen parameters
 - The checkpoint size is reduced from 350GB to 35MB (roughly 10,000×) with rank $r = 4$, and only the query and value projection matrices are adopted

LoRA

- LoRA shows comparable performance compared to fine-tuning (FT) across the entire model, despite tuning $< 0.022\%$ of the parameters
 - The ideal parameter budget varies across datasets
 - On the MNLI-m dataset, LoRA with 4.7M parameters outperforms LoRA with 37.7M parameters

* Accuracy on WikiSQL and MNLI-m datasets after fine-tuning

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m
		Acc. (%)	Acc. (%)
GPT-3 (FT)	175,255.8M	73.8	89.5
GPT-3 (BitFit)	14.2M	71.3	91.0
GPT-3 (PreEmbed)	3.2M	63.1	88.6
GPT-3 (PreLayer)	20.2M	70.1	89.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5
GPT-3 (LoRA)	4.7M	73.4	91.7
GPT-3 (LoRA)	37.7M	74.0	91.6

LoRA

- Adapting LoRA to all weight matrices in the MHA sublayer outperforms other cases
 - We allocate a parameter budget of 18M for all 96 layers of GPT-3 175B.

* Accuracy on WikiSQL and MultiNLI datasets after fine-tuning GPT-3 175B with 18M parameters

	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

- W_q, W_k, W_v , and W_o are query, key, value, and out projection matrices, respectively

LoRA to QLoRA

- LoRA: How to fine-tune a pretrained model quickly?
 - Input: a pretrained model $\mathcal{M}$ (full-precision)
 - Output: a finetuned model $\mathcal{M}$ (full-precision)
- QLoRA: How to fine-tune a pretrained model with less memory overhead?
 - Input: a pretrained model $\mathcal{M}$ (full-precision) and quantization bit B
 - Output: a B -bit quantized model $\hat{\mathcal{M}}$ and adapters (full-precision)

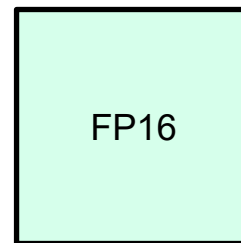
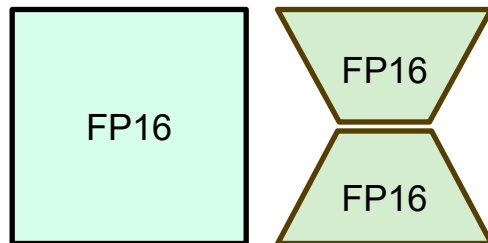
<https://github.com/artidoro/qlora>

QLoRA

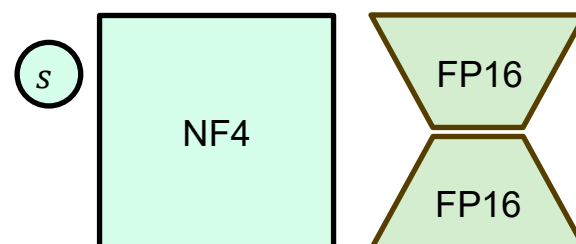
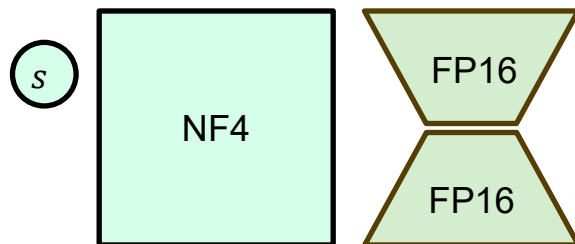
Training

Inference

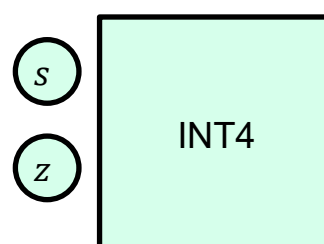
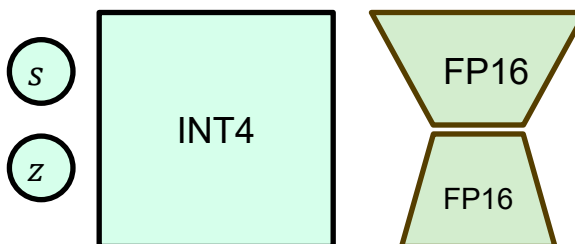
LoRA



QLoRA

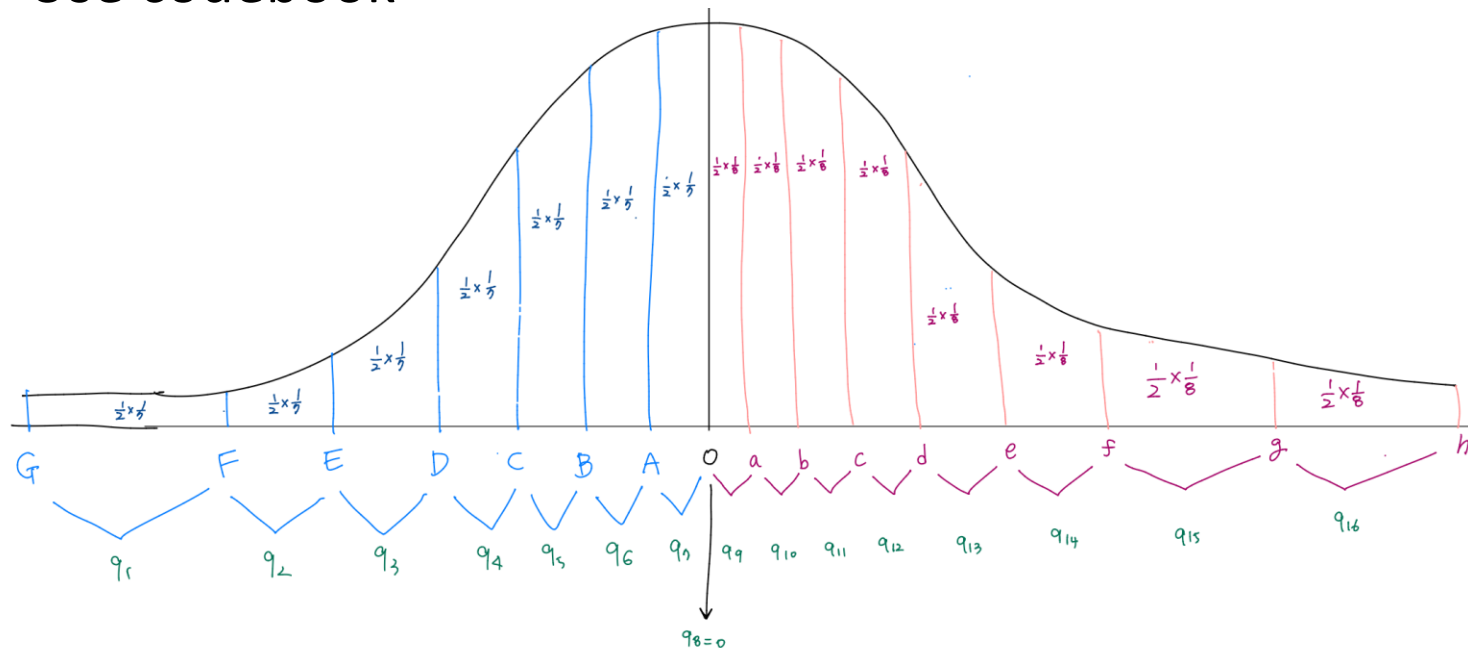


QALoRA



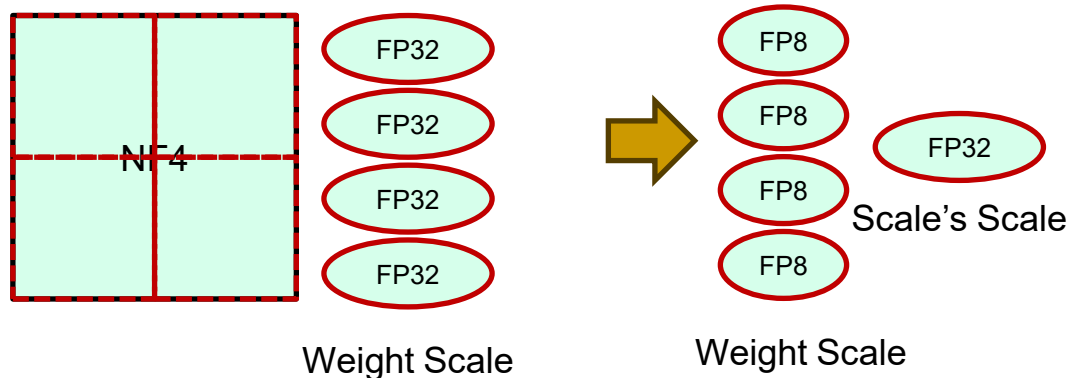
QLoRA (Idea1: 4-bit NormalFloat)

- How to correctly express weights with limited bit widths?
 - Determine quantization levels based on weight distribution (Normal distribution)
 - Use codebook



QLoRA (Idea2: Double Quantization)

- How to decrease the memory for the scale parameters?
 - Each block (64 parameters) requires to store a FP32 scale parameter
 - Double quantization: quantize the scales one more time =>
 - 256 scales are quantized into FP8 with FP32 scale
 - Overhead: $\frac{32}{64} = 0.5 \text{ bits} \rightarrow \frac{8}{64} + \frac{32}{(64*256)} = 0.127 \text{ bits}$



QLoRA to QALoRA

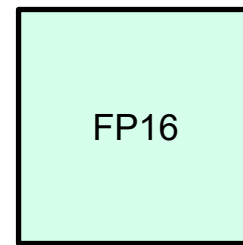
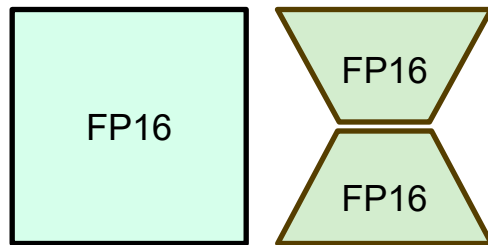
- QLoRA: How to fine-tune a pretrained model with less memory overhead?
 - Input: a pretrained model $\mathcal{M}$ (full-precision) and quantization bit B
 - Output: a B -bit quantized model $\hat{\mathcal{M}}$ and adapters (full-precision)
- QALoRA: How to fine-tune a pretrained model to achieve fast inference with less memory overhead?
 - Input: a pretrained model $\mathcal{M}$ (full-precision) and quantization bit B
 - Output: a B -bit quantized model $\hat{\mathcal{M}}$

QALoRA

Training

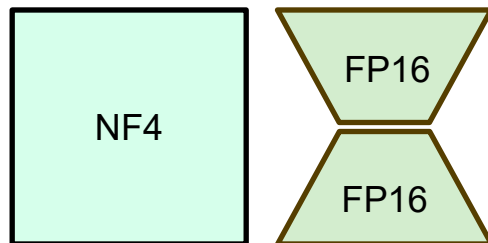
Inference

LoRA

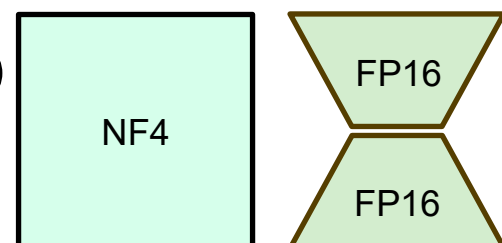


QLoRA

(s)



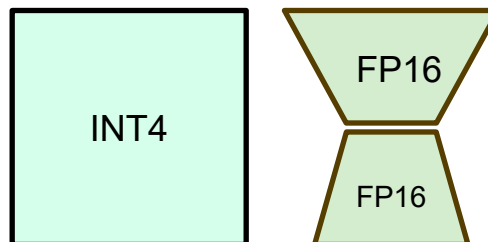
(s)



QALoRA

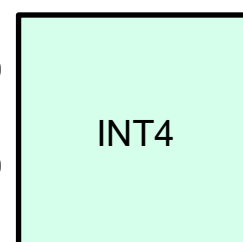
(s)

(z)



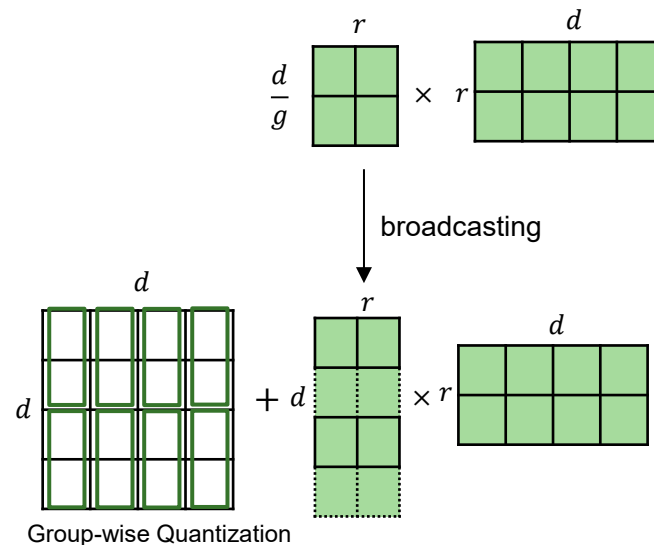
(s)

(z)



QALoRA

- How to remove adapters after QLoRA-training?
 - ❑ Merge LoRA weight to zero-points of quantized weights
 - ❑ One LoRA weight per each quantization group



<https://github.com/yuhuixu1993/qa-lora>

Conclusion

- Language model compression: compress a language model to be fast, memory-efficient, and energy-efficient while maintaining the accuracy
- Quantization method is effective for LLM compression
- PTQ and QAT methods have tradeoff in accuracy and efficiency
- Parameter-efficient fine tuning methods enable effective training

Thank You!